

Using Sum-of-Squares Methods for Lyapunov Function Discovery

Oscar Watkins

August 27, 2026

1 Introduction

Lyapunov functions are used to prove the stability of dynamic systems, without having to solve their underlying differential equations. By representing the “energy” of a system, they guarantee a system will converge towards an equilibrium. Finding Lyapunov functions for a given system has been historically difficult. However, Sum of Squares (SOS) methods can be used to introduce restrictions that allow Lyapunov functions to be discovered. Classical SOS methods are heavily constrained by dimensionality and can only be applied to polynomial systems, limiting their scalability to complex realistic systems. Using recent breakthroughs in AI, symbolic transformer models offer new opportunities for the discovery of Lyapunov functions. Consequently established SOS methods serve as an ideal benchmark to judge the success of these new symbolic transformer models.

In this paper, we apply SOS methods to several benchmarks to evaluate their ability to find global Lyapunov functions. These benchmarks comprise five baseline systems and two datasets of synthetically generated differential equations: FLyap and Poly3_10000.

We then turn to the discovery of Lyapunov functions for power-system dynamics. Because global stability analysis is infeasible for these complex systems, we focus on local Lyapunov functions. We compare the two distinct optimisation algorithms [8][11] for finding Lyapunov functions that maximise the region of attraction (ROA).

2 Theoretical Background

2.1 Lyapunov Functions

In mathematical terms, we consider the dynamical system

$$\dot{x} = f(x), \tag{1}$$

where $x \in \mathbb{R}^n$, $f \in C^1(\mathbb{R}^n)$ and $\dot{x} = \frac{dx}{dt}$. We want to know if the system has a stable equilibrium around a point x^* such that $f(x^*) = 0$. We assume, without loss of generality, that $x^* = 0$.

Definition 2.1. The system (1) is stable when, for any $\varepsilon > 0$, there exists $\eta > 0$ such that, if $\|x(0)\| < \eta$, the system (1) with initial condition $x(0)$ has a unique solution $x \in C^1([0, +\infty))$ and

$$\|x(t)\| \leq \varepsilon, \quad \forall t \in [0, +\infty). \quad (2)$$

In other words, a system is stable if a solution that begins close to the origin ($\|x(0)\| < \eta$) stays close to the origin at all times ($\|x(t)\| \leq \varepsilon$). Lyapunov proved that stability is related to the existence of what is now called a Lyapunov function.

Definition 2.2. The function $V \in C^1(\mathbb{R}^n, \mathbb{R}_+)$ is said to be a global Lyapunov function for the system (1) if the following conditions are satisfied:

$$V(0) = 0, \quad \lim_{\|x\| \rightarrow +\infty} V(x) = +\infty, \quad (3)$$

$$V(x) > 0, \quad \nabla V(x) \cdot f(x) \leq 0 \quad \text{for } x \neq 0. \quad (4)$$

Theorem 2.1 (Lyapunov 1892, [1]). If the system (1) has a Lyapunov function, then it is stable.

Most dynamical systems are unstable. In the general case, there is no systematic way of discovering a Lyapunov function, or even showing that one exists. However, SOS methods can be used to discover functions for small polynomial systems [2].

2.2 Sum Of Squares Theory

SOS methods provide a computational method of discovering Lyapunov functions by replacing positivity conditions with stronger conditions that can be solved using semidefinite programming (SDP). The condition that $p(x)$ is SOS is much more computationally tractable than nonnegativity [3].

Definition 2.3. A polynomial $p(x)$ is called a sum of squares if it can be written in the form

$$p(x) = \sum_{i=1}^m q_i(x)^2, \quad (5)$$

where each $q_i(x)$ is also a polynomial.

If a polynomial $p(x)$ is SOS, then it is non-negative.

The connection to Lyapunov functions comes from the stability conditions: For a polynomial dynamical system find a Lyapunov function $V(x)$ such that $V(x) > 0$ away from the origin and

$$\dot{V}(x) = \nabla V(x) \cdot f(x) \leq 0. \quad (6)$$

If $V(x)$ is chosen to be a polynomial, these conditions become polynomial positivity problems. Instead of trying to prove positivity directly, SOS methods impose the stronger requirements

$$V(x) \text{ is SOS} \quad (7)$$

and

$$-\dot{V}(x) \text{ is SOS.} \quad (8)$$

The first condition ensures that $V(x)$ is non-negative and that $\dot{V}(x)$ is non-positive. If $V(x)$ is zero only at the equilibrium, these conditions ensure that $V(x)$ is a Lyapunov function.

Proposition 2.1. A polynomial $p(x)$ of degree $2d$ is SOS if and only if there exists a positive semidefinite matrix Q and a vector of monomials $Z(x)$ containing monomials in x of degree at most d such that

$$p(x) = Z(x)^T Q Z(x), \quad Q \succeq 0. \quad (9)$$

Proof: [3].

The SDP used for computing the SOS decomposition relies on the Gram matrix method [4]. For Lyapunov discovery, the solver searches for coefficients such that both $V(x)$ and $-\dot{V}(x)$ satisfy the SOS constraints. If successful, it returns a candidate Lyapunov function.

This method is powerful because it turns Lyapunov discovery into a systematic computational problem. However, it has limitations. SOS methods can only be used on polynomial systems to discover polynomial Lyapunov functions. Also, they become increasingly computationally complex as the number of variables or polynomial degree increases. As a result, SOS methods are most effective for relatively small polynomial systems.

3 Benchmark Tests

The experiment was carried out using SOSTOOLS SeDuMi (SDP solver) in MATLAB. The code was based on that from [5].

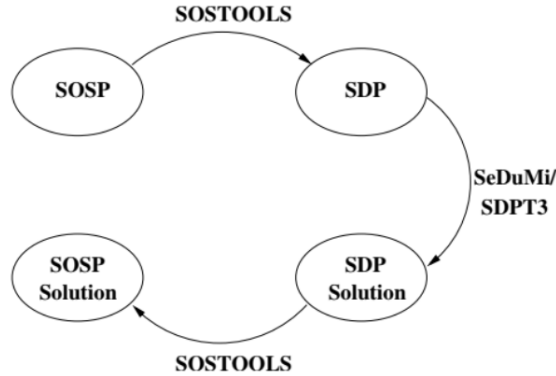


Figure 1: Diagram depicting how SOS programs (SOSPs) are solved with SOSTOOLS. For greater detail [6].

Initially, five different systems of differential equations were chosen. The aim was to compare how SOS methods perform across systems with different levels of complexity. These include a linear

system, a cubic system, a three-dimensional system, a fifth-degree system, and a system with a non-polynomial solution (that should not be solvable). This selection was intended to test both the strengths and the limitations of SOS methods.

Two datasets of differential equations were then used to test the success rate of the SOS solver. FLYap is a forward-generated dataset of polynomial systems with two to three equations, each constructed to have a non-homogeneous polynomial Lyapunov function [2]. The 200 systems used were taken from the FLYap benchmark directory in the Facebook Research Lyapunov repository. For Poly3.10000, 2D and 3D polynomial systems were generated, with a maximum degree of 4. The code was based on that used to construct the Poly3 dataset in [2]. Unlike FLYap, these systems do not necessarily have Lyapunov solutions.

To ensure the discovered candidates were actual Lyapunov functions, they were verified using the Satisfiability Modulo Theory (SMT) solver Z3 in Python. The SMT solver searches for a counterexample that satisfies $\dot{V}(x) > 0$ or $V(x) < 0$. If it is unable to find one, then the candidate function is an actual Lyapunov function [7].

4 Results

4.1 Five Benchmark Systems

Type of System	System Equations	Lyapunov Function Discovered	Verification Successful	Time (s)
Linear system	$\dot{x}_1 = -3x_1 + x_2$ $\dot{x}_2 = -x_1 - 2x_2$	$V(x) = 2.6288x_1^2 - 0.57115x_1x_2 + 3.5522x_2^2$	Yes	0.02878
Cubic system	$\dot{x}_1 = -x_1^3 + x_1^2x_2$ $\dot{x}_2 = -x_1^2x_2 - x_2^3$	$V(x) = 2.9086x_1^2 + 1.1567x_1x_2 + 3.3541x_2^2$	Yes	0.0511
3D system	$\dot{x}_1 = -x_1 - x_2x_3$ $\dot{x}_2 = -x_2 + x_1x_3$ $\dot{x}_3 = -x_3$	$V(x) = 3.3035x_1^2 + 3.3035x_2^2 + 3.3035x_3^2$	Yes	0.1578
Fifth-degree system	$\dot{x}_1 = 6x_1^5 - 2x_1$ $\dot{x}_2 = -x_2^5 - 2x_2$	$V(x) = 0.96759x_1^4 + 2.5625x_1^2x_2^2$ $+ 13.6853x_1^4 + 18.6472x_2^2$	Yes	0.1495
Non-polynomial solution	$\dot{x}_1 = -x_1 + x_1x_2$ $\dot{x}_2 = -x_2$	Unable to find $V(x)$ despite simple solution: $V(x) = \ln(1 + x_1^2) + x_2^2$	N/A	N/A

Summary of Lyapunov function discovery results for different polynomial systems.

The solver was able to successfully find and verify Lyapunov functions for four of the five systems tested. As expected, it failed on the fifth system. Despite the simplicity of the system and its solution, the nature of SOS techniques prevents the discovery of non-polynomial functions. The results also show the effect of polynomial degree on computational cost: the fifth-degree and 3D systems took considerably longer to solve. These outcomes highlight two major limitations of SOS methods that prevent their wider use: they are restricted to polynomial solutions, and their computational cost increases rapidly as the degree of the system grows.

4.2 Generated Datasets Results

FLyap Dataset

Number of Systems	$V(x)$ Not Discovered	Successfully Verified Discovered $V(x)$	Unsuccessfully Verified Discovered $V(x)$	Verification Timeout	Mean Time to Solve and Verify
200	1	170	19	10	2.1997

SOSTOOLS successfully discovered verifiable Lyapunov functions for 85% of the FLyap systems. However, the systems are limited to polynomial expressions and so the results demonstrate little towards the real-world utility of SOSTOOLS.

Poly3_10000 Dataset

Number of Systems	$V(x)$ Not Discovered	Successfully Verified Discovered $V(x)$	Unsuccessfully Verified Discovered $V(x)$	Verification Timeout	Mean Time to Solve and Verify
10000	9982	3	15	0	0.0898

SOSTOOLS was unsuccessful on the Poly3_10000 dataset, discovering a verifiable Lyapunov function for only 3 out of 10,000 systems. The large number of undiscovered candidates highlights a key limitation of SOS methods: they are effective when the problem structure matches polynomial SOS assumptions, but they struggle on broader generated datasets. This is why AI performs better at Lyapunov function discovery [2].

5 Application of SOS Methods to Power System Models

5.1 Background to 6D model

The addition of renewable sources to the electricity grid has made power systems inherently more unstable due to the loss of natural inertia [14]. As a result, proving the stability of power systems with Lyapunov methods has become increasingly important. The 6D model used to test the SOS solver represents a two-machine versus infinite-bus system with transfer conductances. It is taken from [8].

In the original coordinates, the dynamics are

$$\begin{aligned}\dot{x}_1 &= x_2, \\ \dot{x}_2 &= 33.5849 - 1.8868 \cos(x_1 - x_3) - 5.2830 \cos(x_1) \\ &\quad - 16.9811 \sin(x_1 - x_3) - 59.6226 \sin(x_1) - 1.8868 x_2, \\ \dot{x}_3 &= x_4, \\ \dot{x}_4 &= 11.3924 \sin(x_1 - x_3) - 1.2658 \cos(x_1 - x_3) \\ &\quad - 3.2278 \cos(x_3) - 1.2658 x_4 - 99.3671 \sin(x_3) + 48.4810.\end{aligned}\tag{10}$$

where

$$x_1 = \delta_1, \quad x_2 = \omega_1, \quad x_3 = \delta_2, \quad x_4 = \omega_2.$$

This system has a stable equilibrium at

$$x^* = (0.4680, 0, 0.4630, 0).$$

After shifting the variables so that the equilibrium is at the origin, the model becomes

$$\begin{aligned}\dot{x}_1 &= x_2, \\ \dot{x}_2 &= 16.9715 \cos(x_1) \sin(x_3) - 31.6131 \cos(x_1) - 50.8269 \sin(x_1) \\ &\quad - 1.9718 \cos(x_1) \cos(x_3) - 1.8868 x_2 - 16.9715 \cos(x_3) \sin(x_1) \\ &\quad - 1.9718 \sin(x_1) \sin(x_3) + 33.5849, \\ \dot{x}_3 &= x_4, \\ \dot{x}_4 &= 11.3986 \cos(x_3) \sin(x_1) - 47.2723 \cos(x_3) - 87.4618 \sin(x_3) \\ &\quad - 1.2088 \cos(x_1) \cos(x_3) - 11.3986 \cos(x_1) \sin(x_3) - 1.2658 x_4 \\ &\quad - 1.2088 \sin(x_1) \sin(x_3) + 48.4810.\end{aligned}\tag{11}$$

5.2 Recasting Power-System Dynamics

SOS methods can only be applied to polynomial systems. However, models can be transformed into a polynomial system with a larger state dimension [9]. The recasting introduces a number of equality constraints that restrict the states to a manifold with the original state dimension. For the power system introduced earlier, recasting is achieved by a nonlinear change of variables

$$z_{3i-2} = \sin(x_{2i-1}), \quad (12a)$$

$$z_{3i-1} = 1 - \cos(x_{2i-1}), \quad (12b)$$

$$z_{3i} = x_{2i}. \quad (12c)$$

for $i = 1, \dots, n-1$.

Recasting also introduces the equality constraints

$$G_1(z) = z_1^2 + z_2^2 - 2.0z_2 = 0, \quad (13a)$$

$$G_2(z) = z_4^2 + z_5^2 - 2.0z_5 = 0. \quad (13b)$$

The recast dynamics of model B are given by

$$\begin{aligned} \dot{z}_1 &= z_3 - z_2 z_3, \\ \dot{z}_2 &= z_1 z_3, \\ \dot{z}_3 &= 33.6z_2 - 67.8z_1 - 1.89z_3 \\ &\quad + 16.9715z_4 + 1.9718z_5 - 1.9718z_1 z_4 \\ &\quad + 16.9715z_1 z_5 - 16.9715z_2 z_4 - 1.9718z_2 z_5, \\ \dot{z}_4 &= z_6 - z_5 z_6, \\ \dot{z}_5 &= z_4 z_6, \\ \dot{z}_6 &= 11.3986z_1 + 1.2088z_2 - 98.8604z_4 \\ &\quad + 48.4810z_5 - 1.2658z_6 - 1.2088z_1 z_4 \\ &\quad - 11.3986z_1 z_5 + 11.3986z_2 z_4 - 1.2088z_2 z_5. \end{aligned} \quad (14)$$

5.3 Discovering the Lyapunov Function

Using SOSTOOLS, we can search for a solution for $V(x)$ with SOS constraints of the form

$$-s_2(\beta - p) + \dot{\hat{V}} - \lambda_{11}G_1 - \lambda_{12}G_2 - l_1 \text{ is SOS,} \quad (15a)$$

$$-s_6(\beta - p) - \dot{\hat{V}} - \lambda_{21}G_1 - \lambda_{22}G_2 - l_2 \text{ is SOS.} \quad (15b)$$

where

$$p(z) = z_1^2 + z_2^2 + z_3^2 + z_4^2 + z_5^2 + z_6^2, \quad \beta = 0.1,$$

and

$$l_1 = l_2 = 10^{-4}p(z).$$

The functions s_n are SOS polynomials of degrees 0 and 1, and the λ_N terms are free polynomials. See [8] for specific details.

Expression	Value
$\tilde{V}(x)$	$\begin{aligned} \tilde{V}(z) = & 1.0314z_1^2 - 0.25882z_1z_2 + 0.014703z_1z_3 - 0.083709z_1z_4 \\ & - 0.10199z_1z_5 + 0.0067914z_1z_6 + 0.69037z_2^2 - 0.0048537z_2z_3 \\ & - 0.063041z_2z_4 - 0.19715z_2z_5 \\ & - 0.0042701z_2z_6 + 0.020826z_3^2 - 0.0040854z_3z_4 \\ & - 0.00062436z_3z_5 + 0.0059029z_3z_6 + 1.3906z_4^2 \\ & - 0.36484z_4z_5 + 0.008324z_4z_6 + 0.82241z_5^2 \\ & - 0.0052494z_5z_6 + 0.019103z_6^2 + 0.6887z_2 + 0.88519z_5 \end{aligned}$
Lyapunov function recast back into trigonometric form (x variables)	$\begin{aligned} V(x) = & 0.1481 \cos(x_1) \sin(x_1) - 0.0045566x_4 - 1.638991 \cos(x_1) \\ & - 1.735881 \cos(x_3) - 0.206577 \sin(x_1) - 0.204896 \sin(x_3) \\ & - 0.089069 \cos(x_1) \cos(x_3) - 0.0043626x_2 \\ & + 0.030596 \cos(x_1) \sin(x_3) \\ & + 0.058477 \cos(x_3) \sin(x_1) + 0.1743 \cos(x_3) \sin(x_3) \\ & - 0.019781 \sin(x_1) \sin(x_3) + 0.59399 \cos(x_1)^2 + 0.61647 \cos(x_3)^2 \\ & + 0.75621 \sin(x_1)^2 + 0.86356 \sin(x_3)^2 + 0.0046816x_2x_4 \\ & + 0.0026546x_2 \cos(x_1) + 0.001708x_2 \cos(x_3) \\ & + 0.0021184x_4 \cos(x_1) \\ & + 0.0024382x_4 \cos(x_3) + 0.013551x_2 \sin(x_1) \\ & - 0.00080986x_2 \sin(x_3) + 0.0035536x_4 \sin(x_1) \\ & + 0.0087646x_4 \sin(x_3) \\ & + 0.015681x_2^2 + 0.01222x_4^2 + 2.253481 \end{aligned}$

Discovered Lyapunov function

5.4 Estimating Region of Attraction (ROA)

5.4.1 Initial ROA Estimate

In nonlinear dynamical systems, the region of attraction (ROA) is the set of all initial states whose trajectories asymptotically converge to the equilibrium over time. The Lyapunov function found enables us to estimate the ROA for the model. For a scalar c , the Lyapunov sublevel set

$$\Omega_c = \{z \in \mathbb{R}^M \mid \tilde{V}(z) \leq c, G(z) = 0\}$$

provides an invariant estimate of the ROA. Therefore, the best ROA estimate is obtained by finding the largest value of c such that $\Omega_c \subseteq \tilde{D}$.

Here, $\tilde{D}$ is the domain in the recast state space, defined as

$$\tilde{D} = \{z \in \mathbb{R}^M \mid \beta - p(z) \geq 0, G(z) = 0\}.$$

Finding c can be redefined as the following SOS problem [8]:

$$-s_1(c - \tilde{V}) - s_2(p - \beta) - s_3(c - \tilde{V})(p - \beta) - \lambda^T G - (p - \beta)^2 \text{ is SOS.} \quad (16)$$

For the Lyapunov function discovered, the initial estimate of the ROA is very conservative. As shown in Figure 2, the result captures only a small portion of the total attraction region.

5.4.2 Expanding Interior Algorithm

To reduce the conservatism of initial ROA estimates, we can employ an algorithm that systematically enlarges a target region inside the stable domain using a two-tier iterative loop.

Step 1a: the algorithm fixes $\tilde{V} = \tilde{V}^{(i-1)}$ and $\beta = \beta^{(i-1)}$, then uses bisection search to find the largest $c^{(i)}$ while maintaining SOS feasibility.

Step 1b: the algorithm fixes this new $c^{(i)}$ and performs a search on β to find the largest domain $p^{(j-1)}(z) < \beta$ contained inside $\tilde{V}^{(i-1)}(z) < c^{(i)}$.

Step 2a: the algorithm fixes $c = c^{(i)}$ and $\beta = \beta^{(i)}$ and searches for a Lyapunov function such that the region $p^{(j-1)}(z) < \beta$ is included in $\tilde{V}(z) < c$.

Step 2b: $c^{(i)}$ is kept fixed whilst searching to maximise β . The inner loop repeats Steps 1a–2b until the increase in β is below a chosen tolerance.

Step 3: the shape of the polynomial is then updated by setting $p^{(j)} = \tilde{V}^{(i)}$. It then repeats steps 1a to 2b, starting with the new shape for $p(z)$. With each iteration, the outer loop checks the coefficient change between $p^{(j)}(z)$ and $p^{(j-1)}(z)$; if the change is below tolerance, the algorithm stops. The final sublevel set $\tilde{D}^{(i)} = \{z \in \mathbb{R}^M \mid \tilde{V}^{(i)}(z) \leq c^{(i)}\}$ is the resulting ROA estimate.

5.4.3 ROA Results

The expanding interior-point algorithm returns the following Lyapunov function:

$$\begin{aligned}
V(x) = & 0.23455 \cos(x_1) \sin(x_1) - 0.0024486x_4 - 0.82786 \cos(x_1) - 1.47012 \cos(x_3) \\
& - 0.260309 \sin(x_1) - 0.3218963 \sin(x_3) - 0.45566 \cos(x_1) \cos(x_3) - 0.0052896x_2 \\
& + 0.0010963 \cos(x_1) \sin(x_3) + 0.025759 \cos(x_3) \sin(x_1) + 0.3208 \cos(x_3) \sin(x_3) \\
& - 0.27942 \sin(x_1) \sin(x_3) + 0.64176 \cos(x_1)^2 + 0.96289 \cos(x_3)^2 \\
& + 0.6574 \sin(x_1)^2 + 1.0195 \sin(x_3)^2 + 0.0024638x_2 \cos(x_1) + 0.0028258x_2 \cos(x_3) \\
& - 0.002373x_4 \cos(x_1) + 0.0048216x_4 \cos(x_3) + 0.010645x_2 \sin(x_1) - 0.0016688x_2 \sin(x_3) \\
& + 0.0070545x_4 \sin(x_3) + 0.01081x_2^2 + 0.010773x_4^2 + 1.14899.
\end{aligned}$$

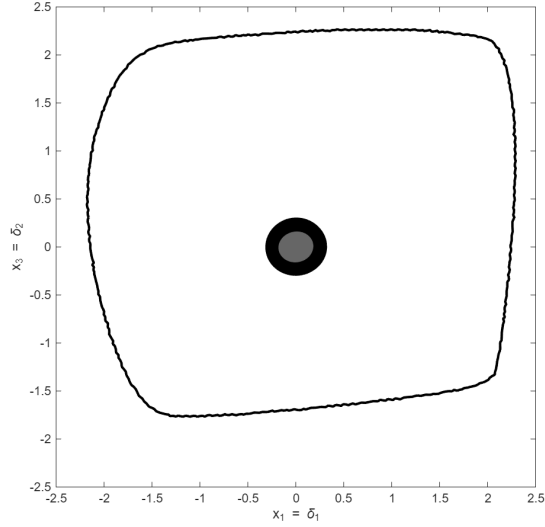


Figure 2: ROA estimate before expanding interior algorithm. (at $\omega_1 = \omega_2 = 0$)

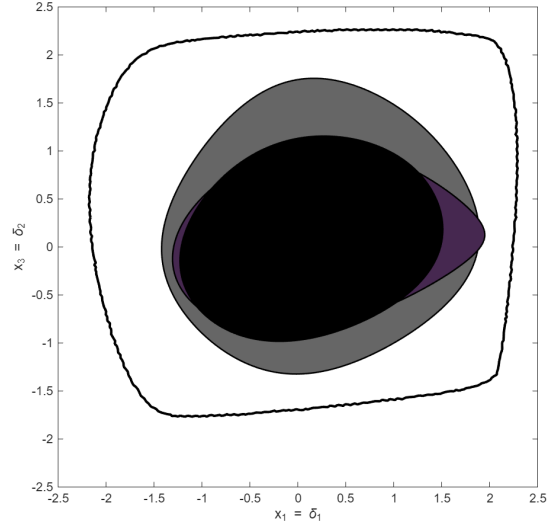


Figure 3: ROA estimate after expanding interior algorithm. (at $\omega_1 = \omega_2 = 0$)

In Figures 2 and 3, the thin black line is the ROA for the SEP located at the origin. The black area represents the ROA estimate for the discovered Lyapunov function. The grey area represents the ROA estimate for $V(x)$, discovered using the same method, from [8]. The purple area represents the estimated ROA calculated using a different method from [10].

As shown in Figures 2 and 3, the algorithm expanded the ROA, but not as successfully as [8]. This could be down to the decision to use lower-degree polynomials, which reduces computational complexity but makes the estimate more conservative.

6 Improving ROA algorithm

Although successful, the expanding interior algorithm used in Chapter 6 is inefficient. Depending on the parameters, expanding the ROA for the simple two-machine versus infinite-bus system took between 10 and 20 minutes. This computational complexity restricts the algorithm's application to more realistic, higher-dimensional power systems.

6.1 Efficient ROA Method

Liu et al. [11] propose a simplified expanding interior algorithm that reduces the computational cost, although it may produce a more conservative ROA estimate than the two-tier iterative method proposed by Anghel et al. [8]. Instead of using nested loops, the method employs a single loop that alternates between optimizing the multiplier polynomials and updating the Lyapunov function. The main steps are as follows:

Step 1: The initial SOS problem is solved to obtain the unnormalised baseline Lyapunov function $V_{0a}(z)$.

Step 2: The function $V_{0a}(z)$ is used to determine the largest stable boundary value, γ_{0b} . The Lyapunov function is then normalised as

$$V_{0b}(z) = \frac{V_{0a}(z)}{\gamma_{0b}}, \quad (17)$$

so that its target boundary is fixed at $V_{0b}(z) = 1$.

Step 3: At iteration i , $V_1^{(i)}(z)$ is set equal to $V_{0b}(z)$. This Lyapunov function is held fixed while the SOS problem is solved to obtain the multiplier polynomials $s_{10(1a)}$ and $s_{11(1a)}$.

Step 4: The multipliers are held fixed while the SOS problem is solved to obtain an expanded Lyapunov function candidate $V_1^{(i+1)}(z)$. If the SOS problem is infeasible, the iteration is terminated and the algorithm proceeds to Step 5. Otherwise, the function is updated according to

$$V_{1b}(z) = V_1^{(i+1)}(z), \quad i \leftarrow i + 1, \quad (18)$$

and the algorithm returns to Step 3.

Step 5: The final feasible Lyapunov function, $V_{1b}(z) = V_1^{(i)}(z)$, is returned. The resulting ROA estimate is the sublevel set

$$\Omega = \{z \mid V_{1b}(z) \leq 1\}. \quad (19)$$

6.2 Efficient ROA Method Results

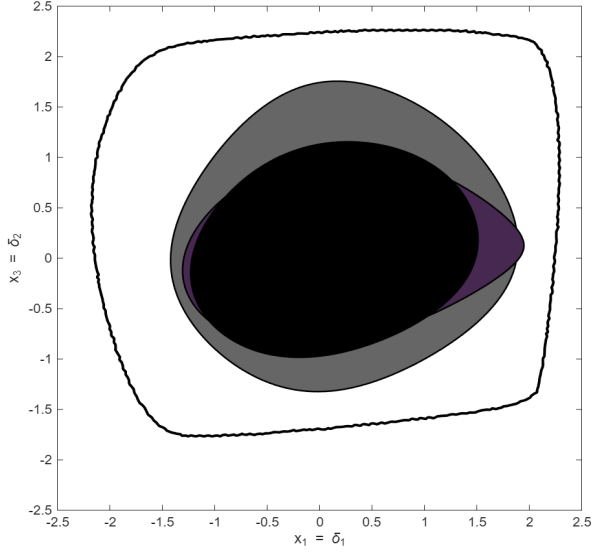


Figure 4: ROA estimate using the algorithm from [8].

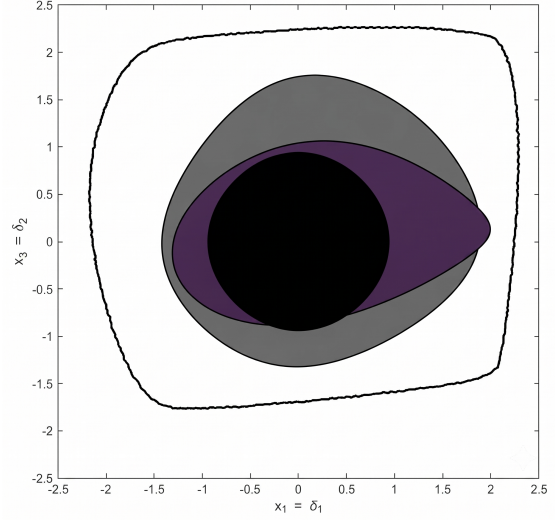


Figure 5: ROA estimate using the algorithm from [11].

As shown in Figures 4 and 5, the algorithm from [11] was less successful at expanding the ROA for the two-machine versus infinite-bus system. However, depending on the chosen parameters, the algorithm from [11] typically completed the expansion in under three minutes, whereas the original algorithm required between 10 and 20 minutes. This increased efficiency may produce a more conservative ROA estimate, but it makes the algorithm more practical for higher-dimension and therefore more realistic power-system models.

7 Application to a Higher Degree System

7.1 Nine-Bus AC/DC System with Wind Farm

To test the capabilities of SOS methods, the algorithm from [11] was applied to a 10th-order AC/DC power system with wind generation, influenced by Case 2 from [12]. Unspecified network parameters were completed using the reduced 9-bus system topology referenced in Mishra et al. [13]. The non-polynomial system dynamics were transformed using Taylor series expansion and then were centred at the origin.

The system dynamics are defined by the following functions.

$$\begin{aligned}
f_1(x) &= x_2, \\
f_2(x) &= -0.1667x_2 - 0.1333x_3^2 - 0.1667x_3\sqrt{x_7^2 + x_8^2} \\
&\quad \times [1.1 \sin(x_1 - \text{atan2}(x_8, x_7)) + 0.12 \cos(x_1 - \text{atan2}(x_8, x_7))] \\
&\quad - 0.1667x_3 (1.2 \sin x_1 + 0.1 \cos x_1) + 0.3916, \\
f_3(x) &= -x_3 + 0.8 \cos x_1 + 0.4060 \\
f_4(x) &= -0.3545x_4 - 6.962 (0.02x_1 + 0.05x_7 + 0.95) \cos x_6 \\
&\quad + 6.948 (0.05x_1 + 0.02x_7 + 0.98) \cos x_5, \\
f_5(x) &= 2.965x_4 - 0.6962 (0.02x_1 + 0.05x_7 + 0.95) \cos x_6 \\
&\quad + 0.6948 (0.05x_1 + 0.02x_7 + 0.98) \cos x_5 - 2.098, \\
f_6(x) &= 0.03545x_4 - 1.1x_6 + 0.6962 (0.02x_1 + 0.05x_7 + 0.95) \cos x_6 \\
&\quad - 0.6948 (0.05x_1 + 0.02x_7 + 0.98) \cos x_5 + 0.3456. \\
f_7(x) &= -319.0x_7 + 314.2x_8(1.0 - 1.0x_9) \\
&\quad - 2.357\sqrt{x_7^2 + x_8^2} \cos(0.1x_1 - \text{atan2}(x_8, x_7)) + 3184.0, \\
f_8(x) &= 2.614x_{10} - 314.2x_7(1.0 - 1.0x_9) - 3141.0x_8 + 501.7, \\
f_9(x) &= -0.1139x_7(-1.25x_{10} - 0.3289x_8) \\
&\quad + 0.1139x_8 \left[1.127x_7 - 1.127\sqrt{x_7^2 + x_8^2} \cos(0.1x_1 - \text{atan2}(x_8, x_7)) \right] + 0.1762, \\
f_{10}(x) &= 0.1295x_7(-1.25x_{10} - 0.3289x_8) \\
&\quad - 0.1295x_8 \left[1.127x_7 - 1.127\sqrt{x_7^2 + x_8^2} \cos(0.1x_1 - \text{atan2}(x_8, x_7)) \right] \\
&\quad - 1.033x_9 + 1.040.
\end{aligned}$$

7.1.1 Transformed System

After applying the Taylor-series transformation and centring the system at the origin, the dynamics are described by the following ten functions.

$$\begin{aligned}
g_1(x) &= x_2, \\
g_2(x) &= 0.137652107649911x_1^2 - 0.253396377841161x_1x_3 - 0.0988985504983511x_1x_7 \\
&\quad - 0.140971601716711x_1x_8 - 0.236608867809184x_1 - 0.166666666666667x_2 \\
&\quad - 0.133333333333333x_3^2 - 0.150973602909463x_3x_7 + 0.105915448994218x_3x_8 \\
&\quad - 0.543837178366610x_3 - 0.140971601716711x_7 + 0.0988985504983511x_8, \\
g_3(x) &= -0.263882050786231x_1^2 - 0.601219637978844x_1 - x_3, \\
g_4(x) &= -0.204645205861732\frac{\sqrt{2}}{\pi}x_1x_5 + 0.0955791724634561\frac{\sqrt{2}}{\pi}x_1x_6 + 0.449875238162639\frac{\sqrt{2}}{\pi}x_1 \\
&\quad + \left(-0.3125 - \frac{0.132}{\pi}\right)x_4 - 7.75821090818209\frac{\sqrt{2}}{\pi}x_5^2 - 0.0818580823446929\frac{\sqrt{2}}{\pi}x_5x_7 \\
&\quad - 4.26774684176673\frac{\sqrt{2}}{\pi}x_5 + 7.48279292500592\frac{\sqrt{2}}{\pi}x_6^2 + 0.238947931158640\frac{\sqrt{2}}{\pi}x_6x_7 \\
&\quad + 4.86262576337214\frac{\sqrt{2}}{\pi}x_6 - 0.437789496309903\frac{\sqrt{2}}{\pi}x_7 - 0.218575 - \frac{0.0923260800000001}{\pi} \\
&\quad + 0.550835966352347\frac{\sqrt{2}}{\pi}, \\
g_5(x) &= -0.0204645205861732\frac{\sqrt{2}}{\pi}x_1x_5 + 0.00955791724634561\frac{\sqrt{2}}{\pi}x_1x_6 + 0.0449875238162639\frac{\sqrt{2}}{\pi}x_1 \\
&\quad + \left(2.96875 - \frac{0.0132}{\pi}\right)x_4 - 0.775821090818209\frac{\sqrt{2}}{\pi}x_5^2 - 0.00818580823446929\frac{\sqrt{2}}{\pi}x_5x_7 \\
&\quad - 0.426774684176673\frac{\sqrt{2}}{\pi}x_5 + 0.748279292500592\frac{\sqrt{2}}{\pi}x_6^2 + 0.0238947931158640\frac{\sqrt{2}}{\pi}x_6x_7 \\
&\quad + 0.486262576337214\frac{\sqrt{2}}{\pi}x_6 - 0.0437789496309903\frac{\sqrt{2}}{\pi}x_7 - 0.0218575 - \frac{0.009232608000000001}{\pi} \\
&\quad + 0.0550835966352345\frac{\sqrt{2}}{\pi}, \\
g_6(x) &= 0.0204645205861732\frac{\sqrt{2}}{\pi}x_1x_5 - 0.00955791724634561\frac{\sqrt{2}}{\pi}x_1x_6 - 0.0449875238162639\frac{\sqrt{2}}{\pi}x_1 \\
&\quad + \left(0.03125 + \frac{0.0132}{\pi}\right)x_4 + 0.775821090818209\frac{\sqrt{2}}{\pi}x_5^2 + 0.00818580823446929\frac{\sqrt{2}}{\pi}x_5x_7 \\
&\quad + 0.426774684176673\frac{\sqrt{2}}{\pi}x_5 - 0.748279292500592\frac{\sqrt{2}}{\pi}x_6^2 - 0.0238947931158640\frac{\sqrt{2}}{\pi}x_6x_7 \\
&\quad + \left(-1.1 - 0.486262576337214\frac{\sqrt{2}}{\pi}\right)x_6 + 0.0437789496309903\frac{\sqrt{2}}{\pi}x_7 \\
&\quad - 0.0550835966352345\frac{\sqrt{2}}{\pi} + \frac{0.009232608000000001}{\pi} + 0.0218575, \\
g_7(x) &= 0.0120410972556147x_1^2 + 0.0200227435591247x_1x_7 - 0.234891363113306x_1x_8 \\
&\quad - 0.0217943960389921x_1 - 3141.58148149585x_7 - 314.159x_8x_9 \\
&\quad - 63.0320274355912x_8 - 56.19676192x_9, \\
g_8(x) &= 2.61380288x_{10} + 314.159x_7x_9 + 62.8318x_7 - 3140.90215713684x_8 + 317.30059x_9, \\
g_9(x) &= 0.000117319584267450x_1^2 + 0.000195086867947595x_1x_7 - 0.00347570900415837x_1x_8 \\
&\quad - 0.000212348545018389x_1 + 0.142369020501139x_{10}x_7 + 0.143792710706150x_{10}
\end{aligned}$$

$$+ 0.0379295189157836x_7x_8 - 0.174095008203042x_7 - 0.010906019004226x_8^2 \\ + 0.0344070767459896x_8,$$

$$g_{10}(x) = -0.000133364581094764x_1^2 - 0.000221767564071902x_1x_7 + 0.00395105794349024x_1x_8 \\ + 0.000241390002609457x_1 - 0.161839857331255x_{10}x_7 - 0.163458255904568x_{10} \\ - 0.0431168797001344x_7x_8 + 0.197904791298600x_7 + 0.012397560603304x_8^2 \\ - 0.0391126972156977x_8 - 1.03342105263158x_9.$$

7.2 Results

The algorithm from [11] resulted in the following Lyapunov function:

$$V(x) = 0.0011273x_1^2 + 0.0013367x_1x_{10} - 0.00048048x_1x_2 + 0.0037826x_1x_3 \\ - 0.00055548x_1x_4 - 0.0019361x_1x_5 + 0.0020405x_1x_6 - 1.1573 \times 10^{-6}x_1x_7 \\ - 1.2344 \times 10^{-5}x_1x_8 + 0.0024543x_1x_9 + 0.021425x_{10}^2 + 2.6587 \times 10^{-5}x_{10}x_2 \\ + 0.0024731x_{10}x_3 - 4.2847 \times 10^{-5}x_{10}x_4 - 0.00043679x_{10}x_5 \\ + 0.00010529x_{10}x_6 - 7.1583 \times 10^{-5}x_{10}x_7 - 0.0020391x_{10}x_8 \\ + 0.18248x_{10}x_9 + 0.0011183x_2^2 - 0.0001616x_2x_3 - 0.00076473x_2x_4 \\ + 0.00059843x_2x_5 - 0.0036852x_2x_6 + 1.4898 \times 10^{-6}x_2x_7 + 3.7735 \times 10^{-7}x_2x_8 \\ - 0.0053736x_2x_9 + 0.0035312x_3^2 - 0.001475x_3x_4 - 0.00056837x_3x_5 \\ - 0.0065726x_3x_6 - 1.71 \times 10^{-6}x_3x_7 - 2.1658 \times 10^{-5}x_3x_8 + 0.0026059x_3x_9 \\ + 0.016894x_4^2 - 0.0025325x_4x_5 + 0.13428x_4x_6 - 2.1287 \times 10^{-6}x_4x_7 \\ - 7.8394 \times 10^{-6}x_4x_8 + 0.00085312x_4x_9 + 0.0074927x_5^2 - 0.053062x_5x_6 \\ + 4.2505 \times 10^{-7}x_5x_7 + 8.8803 \times 10^{-6}x_5x_8 - 0.0026237x_5x_9 \\ + 0.72882x_6^2 - 1.1259 \times 10^{-6}x_6x_7 + 1.6947 \times 10^{-5}x_6x_8 + 0.0085999x_6x_9 \\ + 0.00034263x_7^2 + 1.7166 \times 10^{-5}x_7x_8 - 0.0002991x_7x_9 \\ + 4.9359 \times 10^{-5}x_8^2 - 0.0088014x_8x_9 + 0.40071x_9^2.$$

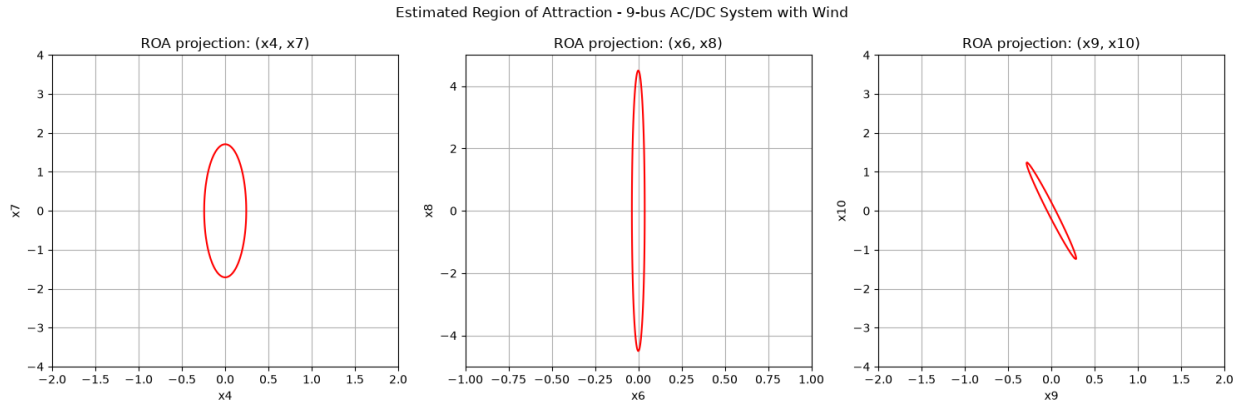


Figure 6: Projected ROA estimates for the nine-bus AC/DC system with wind generation.

As shown in Figure 6, the ROA is fairly limited. This is largely due to computational difficulties. SDP solvers take significantly longer when the degree of the system is increased. As a result, fewer iterations of the algorithm were used for the nine-bus system than for the four-bus system.

8 Conclusion

In this work, we evaluated Sum-of-Squares (SOS) optimisation methods for discovering global and local Lyapunov functions, establishing a benchmark for evaluating emerging AI techniques. Using SOS methods, we demonstrated that SOS programming can provide formal, mathematically guaranteed stability proofs for polynomial dynamical systems. However, SOS techniques are still very limited. They work only on polynomial systems and can produce only polynomial solutions. In addition, they are computationally feasible only for low-dimensional, low-degree systems.

We can extend these methods to power-system dynamics and the investigation of local stability and the Region of Attraction (ROA). Comparing the optimisation approaches of [8] and [11] demonstrated that, while SOS algorithms successfully maximise ROA bounds for smaller networks, scaling to higher-dimensional systems such as the nine-bus model exposes computational bottlenecks. The rapid growth of the Gram matrix Q in the underlying semidefinite program, combined with higher polynomial degrees, dramatically increased solver computation times, directly restricting the size of the discovered ROA.

Ultimately, these results illustrate the trade-off inherent to classical SOS methods: while they offer mathematical rigour and automated discovery, their reliance on polynomial dynamics and vulnerability to the curse of dimensionality limit their applicability to complex, high-dimensional physical infrastructure. These limitations underscore the need for AI-driven techniques [2]. However, there is still a place for using SOS methods as mathematically rigorous verification of AI-discovered Lyapunov functions.

References

1. A. M. Lyapunov, *The general problem of the stability of motion*, PhD thesis, Kharkov University, 1892. <https://www.researchgate.net>.
2. A. Alfarano, F. Charton, and A. Hayat, *Global Lyapunov functions: a long-standing open problem in mathematics, with symbolic transformers*, Oct. 10, 2024, arXiv:2410.08304.
3. P. A. Parrilo, *Structured Semidefinite Programs and Semialgebraic Geometry Methods in Robustness and Optimization*, PhD thesis, California Institute of Technology, Pasadena, CA, 2000. <https://www.mit.edu/~parrilo/pubs/files/thesis.pdf>.
4. V. Powers and T. Wörmann, *An algorithm for sums of squares of real polynomials*, communicated by M.-F. Roy, received Sept. 25, 1995. <https://www.sciencedirect.com>.
5. A. Papachristodoulou and S. Prajna, *SOSTOOLS: Sum of Squares Optimization Toolbox for MATLAB*, 2021. <https://arxiv.org/pdf/1310.4716>.
6. A. Papachristodoulou and S. Prajna, *A tutorial on sum of squares techniques for systems analysis*, in Proceedings of the 2005, American Control Conference, 2005., Portland, OR, USA: IEEE, 2005, pp. 2686–2700. doi: 10.1109/ACC.2005.1470374.

7. D. Ahmed, A. Peruffo, and A. Abate, *Automated and Sound Synthesis of Lyapunov Functions with SMT Solvers*, Department of Computer Science, University of Oxford, Oxford, UK, and Amazon Inc., London, UK, 2020. arXiv:2007.10865.
8. M. Anghel, F. Milano, and A. Papachristodoulou, “Algorithmic construction of Lyapunov functions for power system stability analysis,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 60, no. 9, pp. 2533–2546, Sept. 2013, IEEE:6469186.
9. M. A. Savageau and E. O. Voit, “Recasting nonlinear differential equations as S-systems: A canonical nonlinear form,” *Mathematical Biosciences*, vol. 87, no. 1, pp. 83–115, Nov. 1987.
10. N. G. Bretas and L. F. C. Alberto, “Lyapunov Function for Power Systems With Transfer Conductances: Extension of the Invariance Principle,” *IEEE Transactions on Power Systems*, vol. 18, no. 2, p. 769, May 2003. IEEE:1198313.
11. Y. Liu, H. Yao, Z. Chen, X. Pei, Y. Yang, and Q. Wu, “Estimating the region of attraction of wind integrated power systems based on improved expanding interior algorithm,” *IET Generation, Transmission & Distribution*, vol. 18, no. 12, pp. 2242–2257, 2024. doi: 10.1049/gtd2.13201.
12. Z. Xie, Z. Du, W. Zhou, and W. Zhang, “A Lyapunov approach based on gravitational search algorithm for transient stability assessment of AC/DC systems with wind power,” *IET Generation, Transmission & Distribution*, vol. 17, no. 12, pp. 2739–2756, 2023; received Dec. 22, 2022. doi: 10.1049/gtd2.12871.
13. C. Mishra, R. S. Biswas, A. Pal, and V. A. Centeno, “Critical clearing time sensitivity for inequality constrained systems,” *IEEE Transactions on Power Systems*, vol. 35, no. 2, pp. 1572–1583, Mar. 2020. doi: 10.1109/TPWRS.2019.2942740.
14. J. Leslie, “Energy Explained: How inertia keeps the lights on,” National Energy System Operator (NESO). Accessed: Jun. 3, 2026. [Online]. Available: <https://www.neso.energy/news/energy-explained-how-inertia-keeps-lights>.